

Scalable Latent-Agent Market Simulator

Centralized Market Interpretation, Distributed Participant Cores, and Multi-Timescale Adaptation

BigAlpha 2026 Global University League
AI Open Innovation Challenge

Item	Details
Track	AI Open Innovation Challenge
Primary direction	AI tools and infrastructure / market simulation
Author	Yunfan ‘Damien’ Wang – Carnegie Mellon University*
Notebook	initial_training_optiver_dataset_design_aligned.ipynb
Prototype configuration	Configurable Transformer; current K=24 recurrent agents; one-second decision interval
Project type	Generative market-state simulator, not a signal or factor-mining system

Prototype measurements support feasibility. The architecture and learning philosophy are the primary contribution.

Abstract

This project proposes a market simulator rather than a trading-signal engine. Its purpose is to generate plausible future market states from a changing population of transaction participants. The output is not primarily buy, sell, hold, or an alpha factor. It is a probabilistic market transition: price, spread, imbalance, depth, trade size, and trade count at the next decision step.

The architecture separates market interpretation from participant behavior. A centralized, configurable Transformer first converts user-specified information sources into an implicit market state. Market data, news, sentiment, macro events, cross-asset context, or proprietary firm inputs may be included. The resulting latent representation is not defined through human factor labels. It is learned as an internal digitalization of the information environment, optimized only for usefulness to the downstream participant models.

K independent recurrent decision cores then represent latent participant portraits. Strategy, capital size, reaction speed, risk tolerance, holding horizon, and liquidity preference are not assigned manually in the prototype; they emerge through separate recurrent memories, stochastic action policies, and population weights. The agents do not converse at the same time step. They interact through the market outcome that their aggregated actions produce.

Learning occurs on three timescales. Structural encoder retraining is infrequent and expensive, using the latest approved full dataset when the information regime has materially changed. Participant cores are recalibrated more frequently, potentially daily, because the active strategy mixture and participant size vary faster. Bounded on-board fine-tuning handles intraday deviations through small, reversible updates. One system. Three clocks.

The design also targets a central engineering weakness of many current multi-agent proposals. End-to-end LLM agent systems repeatedly process text, require externally engineered memory, and often coordinate through expensive natural-language exchanges. Here, the expensive information interpretation is shared once; participant evolution uses compact recurrent states and vectorized numerical actions. The result is a simulator whose principal scaling problem becomes ordinary tensor computation rather than repeated language-model inference.

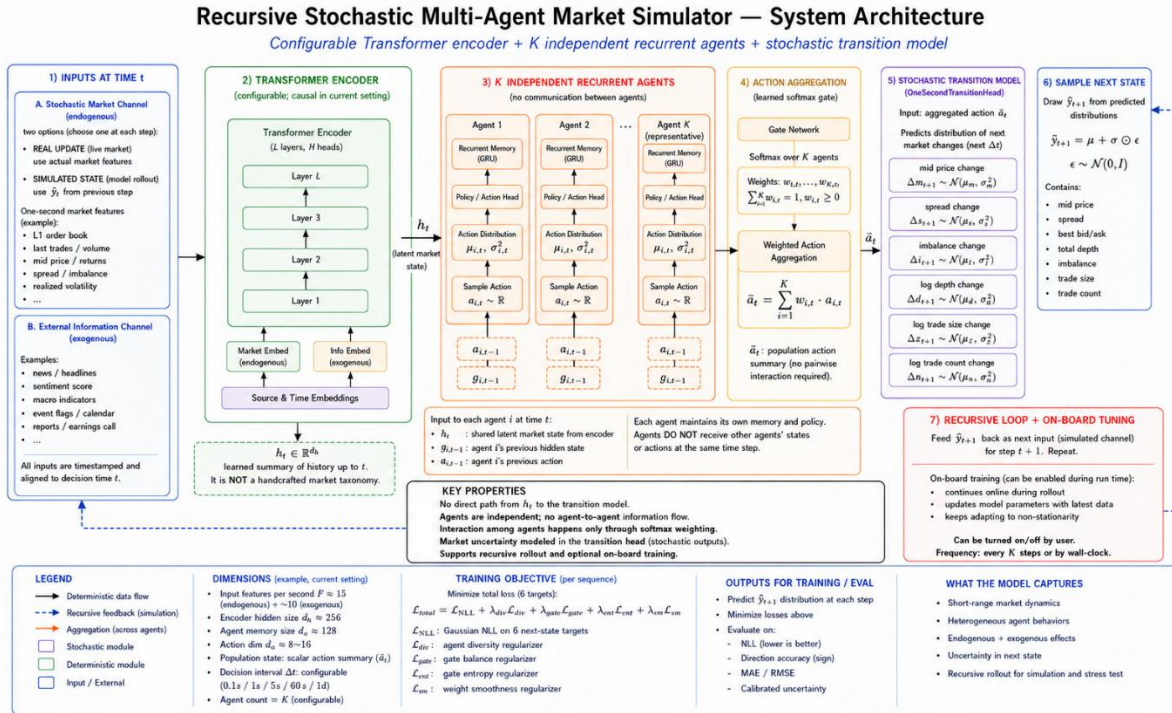


Figure 1. System architecture. The Transformer configuration and K are hyperparameters; participant cores are independent within a decision step.

1. Project thesis: a market simulator, not a trading-signal system

A conventional quantitative model often asks which asset will outperform, which factor contains alpha, or which action maximizes a portfolio objective. This project asks a different question: given the available information and a changing population of transaction participants, what market state could emerge next?

Observed information -> implicit market state -> participant actions -> market transition

The distinction is consequential. A signal model can be successful while remaining silent about the formation of the next market state. A simulator must reproduce the joint movement of several variables and remain coherent when its own outputs are fed back. More demanding. Also more useful for stress testing, execution research, synthetic scenario generation, and evaluation of downstream trading systems.

1.1 What the model generates

- I. A distribution over next-step market returns rather than a single deterministic price forecast.
- II. Changes in spread, order-book imbalance, and top-level market depth.
- III. Trade-size and trade-count dynamics.
- IV. Recursive market paths, produced by reconstructing the next state and returning it to the model.

1.2 What the model does not claim

- 1. It does not identify named real firms from aggregate market data.
- 2. It does not equate directional validation accuracy with trading profitability.
- 3. It does not assume that K equals the true number of market participants.
- 4. It does not require the latent state to correspond to human-readable financial factors.

2. Why existing multi-agent trading architectures are not enough

Recent financial multi-agent systems commonly imitate the organizational chart of an investment firm. One language agent reads fundamentals, another interprets sentiment, another performs technical analysis, and a manager or trader combines their written arguments [1]. This is useful for slow research workflows. It is not the same problem as numerical market simulation.

An end-to-end LLM population faces several engineering constraints when the update interval becomes short. Every decision may require repeated prompt construction and token generation. Persistent state is not inherent in a single generation; memory must be added through context, retrieval, summaries, or an external store [2]. Multi-round discussion introduces more latency. The context grows, or it must be compressed. Reproducibility also becomes difficult when natural-language reasoning changes across prompts, sampling seeds, or model versions.

The objection is not that LLM agents are useless. They are strong candidates for the external-information channel: converting reports, calls, news, or policy documents into timestamped structured inputs. The objection is architectural. Using a language model from perception through action at every market step is unnecessarily expensive for a high-frequency state simulator.

2.1 Architectural comparison

Dimension	Role-based LLM multi-agent system	Proposed latent-agent simulator
Primary output	Textual analysis and trade decision [1]	Probability distribution over the next market state
State persistence	External context, retrieval, or memory layer [2]	Fixed-dimensional recurrent state inside each participant core
Agent communication	Often natural-language debate or role hand-off	No same-step communication; interaction occurs through the market transition
Per-step cost	Repeated LLM inference and token processing	One shared encoder pass plus vectorized recurrent updates
Time resolution	Better suited to slower research cycles	Designed for configurable numerical decision intervals
Reproducibility	Sensitive to prompts, model versions, and sampling	Numerical checkpoints, explicit stochastic distributions, controlled seeds
Scalability lever	Reduce calls, context, or discussion rounds	Parameter sharing, adapters, hierarchy, batching, and state caching

2.2 The scalability argument

The architecture addresses scalability by separating expensive shared interpretation from lightweight participant evolution. Market and external information are encoded once by a centralized Transformer and distributed as a fixed-dimensional latent state to K recurrent participant cores. Each core updates compact memory and emits a numerical action. No text generation. No all-to-all dialogue. Participant-stage computation therefore grows approximately linearly with K and can be vectorized on accelerators. The approach does not eliminate every bottleneck: standard attention remains expensive for long sequences, and fully distinct recurrent cores grow linearly in parameters. Yet the remaining problems are conventional systems problems. Shared backbones, low-rank adapters, hierarchical populations, sparse attention, state caching, and distributed execution can address them. ABIDES shows that engineered market simulators can scale to tens of thousands of agents [3]; our different aim is to learn a compressed behavioral population from historical data while retaining efficient recurrent execution.

3. Core architecture philosophy

3.1 A common information substrate

Transaction parties do not possess identical information. Latency, proprietary feeds, research capability, and private objectives differ. Still, most participants react to a substantially overlapping public substrate: prices, order flow, trades, market events, news, and macro conditions. The centralized encoder represents this shared substrate. Firm-specific or privileged inputs can enter through separate adapters when available.

3.2 Centralized programmable interpretation

Each firm can define a different input schema. One deployment may use only order-book variables. Another may add news embeddings and sentiment. A third may include options, sector flows, or proprietary indicators. The encoder is pretrained to interpret that selected information space and produce a stable latent interface for the participant layer.

$$h_t = E_{\text{theta}}(X_{\text{market},t}, X_{\text{external},t}, M_{\text{time},t})$$

In the current prototype, E_{theta} is a causal Transformer. Causality, depth, attention heads, width, and decision interval are parameters. The architectural requirement is not a specific four-layer model. It is a centralized representation layer that creates one consistent information boundary for the population.

3.3 Implicit state rather than predefined factors

The coordinates of h_t are not named in advance. No forced momentum dimension. No handcrafted panic score. No fixed liquidity regime label. During training, the representation may combine these concepts, or discover structures that have no convenient human description. This is deliberate.

The term implicit market state is preferred to implicit factor. In quantitative finance, factor often implies an explanatory or tradable return factor. Here, the latent state is an internal computational representation. Its validity is judged by downstream participant behavior and transition accuracy, not by whether individual coordinates receive persuasive labels.

3.4 Distributed participant portraits

Each recurrent core represents a latent participant portrait. Strategy family, order size, risk budget, reaction speed, holding horizon, liquidity demand, sensitivity to news, and execution aggressiveness may all influence that portrait. In the prototype they are not explicit metadata fields. They are represented implicitly through separate parameters, hidden states, previous actions, and stochastic policies.

$$g_{i,t} = \text{GRU}_i([h_{t,a_i,t-1}], g_{i,t-1})$$

$$a_{i,t} \sim N(\mu_{i,t}, \sigma_{i,t}^2)$$

Agents do not talk to one another within the same step. Their interaction is market-mediated. Actions combine, the market changes, and the resulting state becomes visible at the next step. This avoids an artificial requirement that every transaction party exchange explanations before acting.

3.5 Action aggregation and transition

$$w_t = \text{softmax}(s_1, t, \dots, s_K, t), \quad a_{\text{bar}_t} = \sum_i w_{i,t} a_{i,t}$$

The current implementation uses a learned gate and weighted action aggregation. The transition model then maps the population action to six next-step distributions. The intended design removes a direct h_t -to-transition shortcut. Otherwise the model could bypass the agents, weakening the central behavioral interpretation. (There would be more mathematical proof if not for the page limit.)

4. Multi-timescale learning

Yet, we need to consider real application scenario and cost of time for quant companies. The architecture separates what changes slowly from what changes quickly. This distinction is central to both compute efficiency and interpretability.

4.1 Structural encoder retraining

Uniquely by this proposal, the centralized encoder captures relationships in the information environment: how the selected feeds interact, what context is useful, and how the market state should be represented for the participant population. These relationships drift, but generally not at the same speed as individual participant behavior. A useful term is systematic representation drift.

Now, the proposed operating policy is infrequent full retraining, perhaps monthly, quarterly, or semiannually depending on the market and data source. Each structural retraining run uses the latest approved complete rolling dataset and repeats the full training pipeline. Daily full retraining is avoided because it is expensive and may make the shared representation unnecessarily unstable. Whether a fresh initialization or warm start is superior remains an empirical question.

4.2 Daily participant-population recalibration

The participant layer changes faster. Capital allocation, active strategy mix, inventory pressure, reaction speed, execution urgency, and liquidity provision can change from day to day. For that, the recurrent decision cores, policy heads, and population weights should be eligible for daily recalibration while the encoder remains fixed.

From a higher level perspective, this does not mean that the system tracks named firms. Without participant-level labels, it estimates a changing latent decomposition of aggregate behavior. And daily retraining therefore updates participant portraits and their relative activity, not a registry of actual organizations.

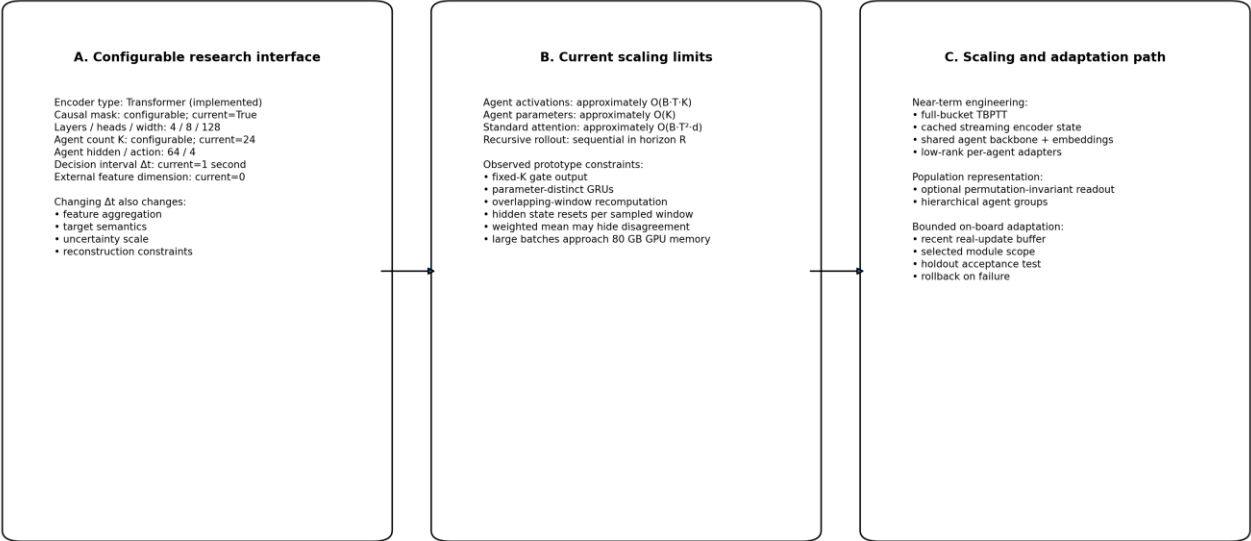
4.3 On-board adaptation

Now, intraday changes may occur between daily training jobs. A news shock, withdrawal of liquidity, rapid change in order size, or altered reaction speed can make the active participant mixture stale. Now, on-board adaptation addresses this gap through bounded local updates.

- I.** Store recent prediction contexts and later real outcomes.
- II.** Build a small calibration batch and a separate local holdout slice.
- III.** Update only approved modules, such as output calibration, the transition head, gating parameters, or small participant adapters.
- IV.** Use a low learning rate, limited update steps, finite-gradient checks, and strict runtime limits.
- V.** Promote the candidate only when the holdout and stability tests improve; otherwise restore the previous checkpoint.

Not continuous unconstrained learning. A controlled adaptation mechanism, with rollback.

Figure 2. Configurability, scalability, and bounded adaptation



Solid items are implemented or directly parameterized in the Notebook. Permutation-aware and hierarchical population readouts remain design extensions.

Figure 2. Configuration, scaling limits, and bounded adaptation path. Implemented parameters are separated from proposed scaling extensions.

5. Prototype implementation

Side by side of the report is the prototype providing evidence that the proposed decomposition can be trained end to end on high-frequency market data. It is not yet the complete product architecture. The external-information channel and production on-board training remain deployment extensions.

Module	Current prototype	Architectural interpretation
Decision interval	One second	Configurable delta-t
Market inputs	19 normalized market features	Endogenous information channel
Encoder	Causal Transformer, 4 layers, 8 heads, d_model=128	Centralized market interpretation
Participant count	K=24	Configurable latent population size
Participant memory	Independent GRUs, hidden size 64	Persistent behavioral state
Action dimension	4 per participant	Compact stochastic decision output
Population mechanism	Softmax gate and weighted action mean	Current aggregation method
Transition targets	6	Reduced next-market-state distribution
Training sequence	96 warmup + 32 supervised positions	Short context followed by next-step supervision

5.1 Dual input channels

Other than idempotent input channels, the endogenous channel accepts a real market update or a simulated state reconstructed from the previous output. A source indicator distinguishes them. The exogenous channel is structurally reserved for timestamped news, sentiment, events, or scenario controls. In the current Optiver prototype, `external_feature_dim=0`; the report does not present external-channel results that have not been implemented.

5.2 Recursive state reconstruction

Left in the tail of the architecture is the transition head predicting next return, spread change, imbalance change, log total-depth change, log trade size, and log trade count. A deterministic reconstruction layer converts these sampled changes into the next reduced market state. That reconstructed state can then re-enter the endogenous channel.

5.3 Numerical engineering

1. Owner will need to validate feature and target schemas before dataset construction.
2. Reject non-finite values and invalid normalization statistics.
3. Evaluate Gaussian negative log-likelihood in float32, even when the forward pass uses bfloat16.
4. Run a forward-and-backward preflight before starting a long GPU job.
5. Terminate immediately on a non-finite loss or gradient.

6. Preliminary training evidence

The uploaded Notebook contains a complete 30-epoch training trace. The objective falls smoothly after the early epochs. Directional validation metrics rise gradually and stabilize near the end of training. At the best validation-loss epoch, balanced accuracy is approximately 58.77%, non-zero direction accuracy approximately 58.88%, F1 approximately 59.53%, and the majority baseline approximately 53.72%.

These values support feasibility. They do not establish a profitable strategy, because the project is not evaluated as one. The stronger standard is whether the model improves against structural baselines and produces stable recursive market paths.

Figure 3. Measured training and validation evidence from the uploaded Notebook

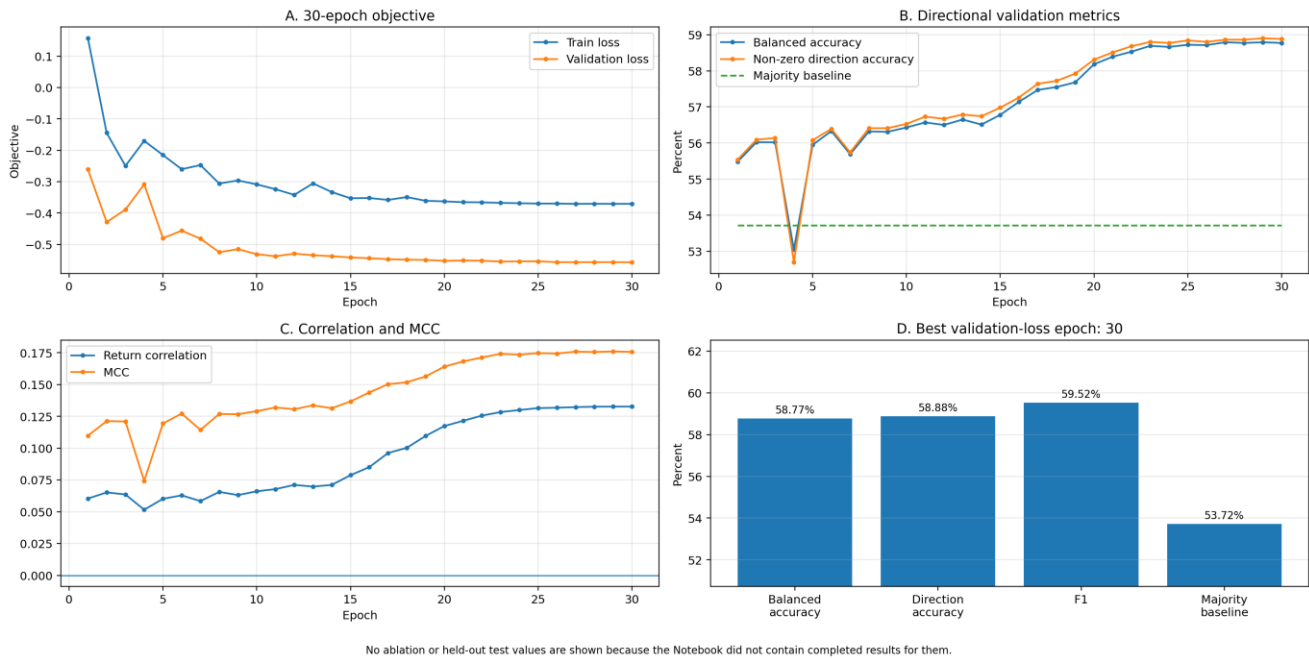


Figure 3. Measured training and validation evidence from the prototype Notebook. No uncompleted ablation or held-out test results are fabricated.

7. Evaluation plan

7.1 Predictive quality

1. Gaussian negative log-likelihood and calibration coverage.
2. Normalized MAE and RMSE.
3. Raw MAE for all six transition targets.
4. Return correlation, balanced accuracy, F1, and Matthews correlation coefficient.

7.2 Participant-population quality

1. Gate entropy and effective active-participant count.
2. Pairwise action correlation and diversity.
3. Weight concentration across regimes.
4. Sensitivity of participant actions to latent-state perturbations.
5. Stability of learned portraits across seeds and retraining windows.

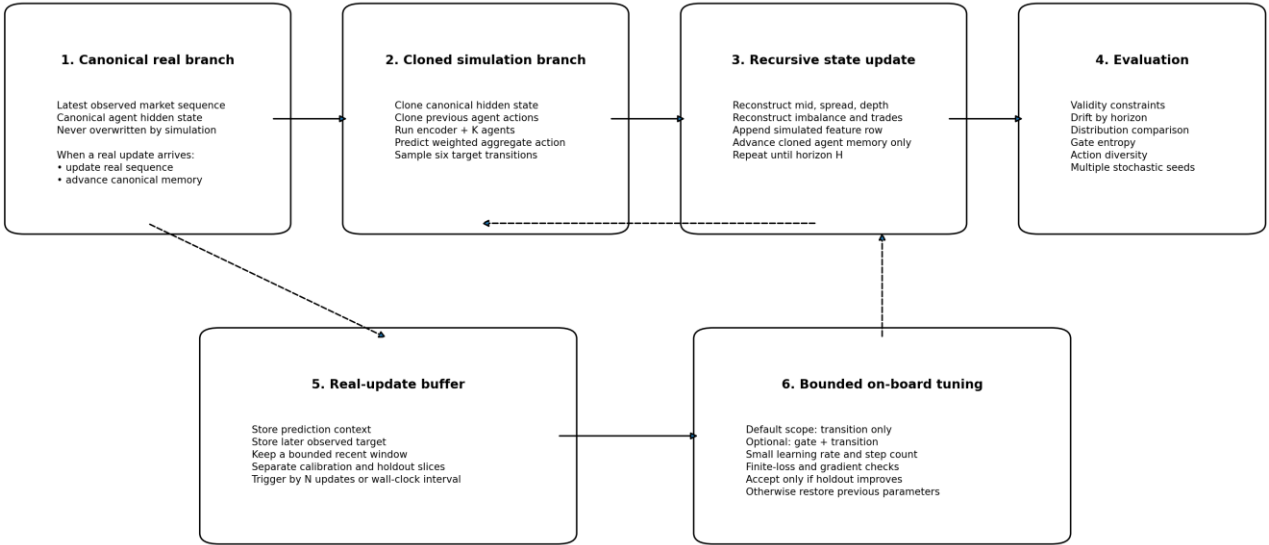
7.3 Simulator quality

1. Fraction of reconstructed states satisfying market validity constraints.
2. Distribution drift as rollout horizon increases.
3. Return, spread, imbalance, depth, trade-size, and trade-count distributions.
4. Autocorrelation and cross-correlation structure.
5. Response to controlled exogenous scenarios after that channel is implemented.

7.4 Required ablations

Ablation	Question answered
Transformer-only predictor	Do participant cores add predictive value?
Single recurrent participant	Does a population improve the result?
Uniform action averaging	Does the learned gate add value?
Different K values	How sensitive is the simulator to population size?
Direct h_t transition input	Does the transition bypass the participant layer?
No daily participant recalibration	Does fast-timescale retraining improve current-regime fit?
No on-board tuning	Does bounded runtime adaptation improve local fit without destabilizing rollout?
Observed-only training	Does exposure to generated states improve recursive robustness?

Figure 4. Recursive simulation, canonical state, and bounded on-board adaptation



This figure is a protocol diagram. The uploaded Notebook does not contain completed recursive-rollout or on-board adaptation measurements, so no synthetic performance curves are shown.

Figure 4. Recursive simulation protocol, canonical state handling, and bounded on-board adaptation. The figure shows the evaluation contract, not invented performance curves.

8. Engineering path to larger populations

The current K=24 model uses parameter-distinct recurrent cores and a fixed-width gate. Vectorization removes Python overhead but does not make the population free. Increasing K raises recurrent activations and parameters approximately linearly. Increasing sequence length raises standard attention cost approximately quadratically.

Current limitation	Engineering response
Parameter-distinct recurrent cores	Shared participant backbone with learned portrait embeddings
Fixed-K gate	Variable-size, permutation-invariant readout
Overlapping-window recomputation	Full-bucket truncated backpropagation and cached encoder state
Long Transformer histories	Sparse, local, linear-time, or streaming attention
Weighted mean hides disagreement	Add dispersion, positive/negative mass, and concentration summaries
Very large participant populations	Hierarchical groups or mixture-of-experts routing
Production latency	Distillation, quantization, and distributed batched execution

The design philosophy remains unchanged as these optimizations are introduced. Shared interpretation. Distributed participant memory. Market-mediated interaction. Multiple adaptation clocks.

9. Platform and commercial relevance

The immediate product is research infrastructure. A user configures the market, decision interval, information sources, participant population, transition schema, and scenario. The system returns possible market paths, uncertainty, and participant-population diagnostics.

User	Research problem	Potential value
Quantitative research team	Historical data contains only realized paths	Generate conditional synthetic scenarios
Execution or market-making team	Liquidity and impact are path-dependent	Stress-test execution logic before deployment
Risk and model-validation team	Rare intraday regimes are underrepresented	Construct adverse spread, depth, and activity paths
Research platform	Users need reusable simulation infrastructure	Offer configurable Notebook, module, or API

The centralized encoder is commercially important because it can be trained around each firm's information schema and then reused across many participant configurations. Expensive structural retraining is periodic. Daily and intraday adaptation are concentrated in smaller modules. This division lowers operational cost and makes model changes easier to govern.

10. Related work and architectural positioning

Reference	Research line	Relationship to this project
[1]	Role-based LLM trading agents	Shows the use of specialized language agents for analysis, debate, risk management, and trade decisions; our system targets numerical market generation instead.
[2]	Memory mechanisms for LLM agents	Supports the observation that persistent agent memory requires explicit context, storage, retrieval, and management mechanisms.
[3]	ABIDES	Demonstrates large-scale discrete-event market simulation and configurable agent interaction.
[4]	Neural stochastic agent-based LOB simulation	Shows a hybrid path between data-grounded stochastic dynamics and explicit trading agents.
[5]	Transformer sequence modeling	Provides the attention architecture used for centralized information interpretation.
[6]	DeepLOB	Provides evidence that learned representations can extract predictive structure from limit-order-book data.
[7]	Transformer models for LOBs	Applies attention-based sequence modeling directly to order-book forecasting.
[8]	Set Transformer	Provides a possible future method for variable-size, permutation-aware population aggregation.

References

- [1] Xiao, Y., Sun, E., Luo, D., and Wang, W. (2024). TradingAgents: Multi-Agents LLM Financial Trading Framework. arXiv:2412.20138.
- [2] Zhang, Z., Bo, X., Ma, C., Li, R., Chen, X., Dai, Q., Zhu, J., Dong, Z., and Wen, J.-R. (2024). A Survey on the Memory Mechanism of Large Language Model Based Agents. arXiv:2404.13501.
- [3] Byrd, D., Hybinette, M., and Balch, T. H. (2019). ABIDES: Towards High-Fidelity Market Simulation for AI Research. arXiv:1904.12066.
- [4] Shi, Z. and Cartlidge, J. (2023). Neural Stochastic Agent-Based Limit Order Book Simulation: A Hybrid Methodology. arXiv:2303.00080.
- [5] Vaswani, A. et al. (2017). Attention Is All You Need. arXiv:1706.03762.
- [6] Zhang, Z., Zohren, S., and Roberts, S. (2018). DeepLOB: Deep Convolutional Neural Networks for Limit Order Books. arXiv:1808.03668.
- [7] Wallbridge, J. (2020). Transformers for Limit Order Books. arXiv:2003.00130.
- [8] Lee, J. et al. (2019). Set Transformer: A Framework for Attention-Based Permutation-Invariant Neural Networks. Proceedings of ICML, PMLR 97.
- [9] BigAlpha 2026 AI Open Innovation Challenge. Organizer-provided competition introduction and submission requirements.

Appendix A. AI application description

Component	Model	Input	Output	Training timescale	Status
Market interpretation	Configurable Transformer; causal in the prototype	Market channel and optional external sources	Implicit market state h_t	Periodic structural retraining	Market channel implemented; external channel planned
Participant memory	K independent GRUs	h_t , own prior state, own prior action	Updated participant states	Daily recalibration plus bounded intraday adaptation	Implemented
Participant action	Gaussian policy heads	Participant state	Action distribution and sample	Daily / intraday	Implemented
Population weighting	Softmax gate	Shared state and participant actions	Weighted population action	Daily / intraday	Implemented
Market transition	Probabilistic six-target head	Weighted population action	Next-state means and uncertainties	Daily or regime-specific	Implemented in aligned Notebook
Recursive rollout	State reconstruction and feedback	Sampled transition plus current state	Next simulated market state	No training during ordinary inference	Implemented
On-board adaptation	Bounded selected-module tuning	Recent prediction contexts and observed outcomes	Accepted or rejected candidate update	Intraday / runtime	Prototype utility implemented; deployment evaluation pending